

NBME - Score Clinical Patient Notes



Identify Key Phrases in Patient Notes from Medical Licensing Exams

[NBME - Score Clinical Patient Notes | Kaggle](#)

比赛介绍

当你去看医生时，他们如何解释你的症状可以决定你的诊断是否准确。学习和评估写病人笔记的技能需要其他医生的反馈，这是一个时间密集的过程，可以通过加入机器学习来改进。

具体来说，你要开发一种自动方法，将考试评分标准中的临床概念（以下称Feature），如“食欲减退”，与医学生写的临床病人笔记（以下称Note）中表达这些概念的各种方式（如“吃得少”、“衣服更宽松”）进行映射。

- 竞赛类型：本次竞赛属于深度学习/自然语言处理，所以推荐使用的模型或者库：**Roberta/DeBERTa**
- 赛题数据：官方提供的训练集大约有 **40,000 份病人笔记**，但其中只有1000份病人笔记有Feature注释。测试集大约有**2,000 份带注释的病人笔记**。这次比赛取得高分的关键是利用3万多份没有注释的笔记做无监督学习。
- 评估标准：我们从所有实例的TPs、FNs和FPs汇总计算出一个**micro-averaged F1 score**。具体我们在讲义1中介绍的非常清楚。官方详见：[NBME - Score Clinical Patient Notes | Kaggle](#)
- 推荐阅读 Kaggle 内的一篇 EDA（探索数据分析）来获取一些预备知识：[NBME Detailed - EDA | Kaggle](#)

数据说明

官方提供的训练集大约有 **40,000 份病人笔记**（归属于10个病历case），但其中只有1000份病人笔记有Feature注释（10个case中每个100份）。测试集大约有**2,000 份带注释的病人笔记**。这次比赛取得高分的关键是利用3万多份没有注释的笔记做无监督学习。

官方数据页面 [NBME - Score Clinical Patient Notes | Kaggle](#)

- **重要术语**
 - **Clinical Case(临床病例)**：病例的种类。本数据集中有十个临床病例。
 - **Patient Note(病人笔记)**：患者在问诊过程中，医学生记录的病人笔记。
 - **Feature(特征)**：临床专业概念。这是我们要预测的标签。可以和病人笔记中的相关表达相互映射。
- **patient_notes.csv**
 - pn_num：每个病人笔记的唯一标识。
 - case_num：一个病人笔记所属于的case的唯一标识。
 - pn_history：病人笔记文本内容。

- **features.csv**
 - feature_num: 每个feature的唯一标识符。
 - case_num: 每个case的唯一标识符。
 - feature_text: 对该feature的描述。
- **train.csv**
 - id: 每个note-feature对的唯一标识符。
 - pn_num: 本样本的病人笔记。
 - feature_num: 本样本的feature。
 - case_num: 该患者笔记所属的case。
 - annotation: 患者笔记中表示feature的文字。一个特征可以在一个笔记中被多次指出。
 - location: 表示每个annotation在笔记中的位置的字符span。可能需要多个span来表示一个注释, 在这种情况下, span由分号分隔。

解决方案思路

在数据上, 由于官方数据的有一些不干净的原标签, 所以我们使用经过修复的train_processed.pkl (已放入datasets目录下)。

其次, **本次比赛的最大特点是**虽然比赛提供了40,000 份病人笔记, 但只有1000份有标注的标签, 所以在我们使用了pseudo label (伪标签) 来训练大量的无标注数据。

模型上我们选择了**DeBERTa系列模型**, 具体使用了 HuggingFace库中的 "**microsoft/deberta-base**", "**microsoft/deberta-large**", "**microsoft/deberta-v3-large**"。

本次比赛因为pseudo label的加入, 所以pipeline相对比较复杂, 具体流程如下:

1. Pretrain 预训练

读取**所有的notes数据** (无论有没有标签), 使用**mlm=0.2**方式对模型进行预训练, 并保存预训练模型。(训练上述DeBERTa系列3个模型)

2. Train 训练

载入上一步预训练的模型, 对有标签的**14,000个样本(包含1000份notes)**进行5Fold训练。(训练上述DeBERTa系列3个模型)

3. Pseudo Label Prediction为无标签样本打上伪标签

使用第2步训练好的模型, 对无标签样本打上pseudo label, 注意5个Fold各自对应的训练集和验证集, **防止数据泄露**。并使用**Blend方法 (讲义4知识点)**融合上述3个模型的预测的pseudo label, 3个模型的加权重分别是base:0.19, large:0.37, v3-large:0.44。

4. All Data Train 全量数据训练

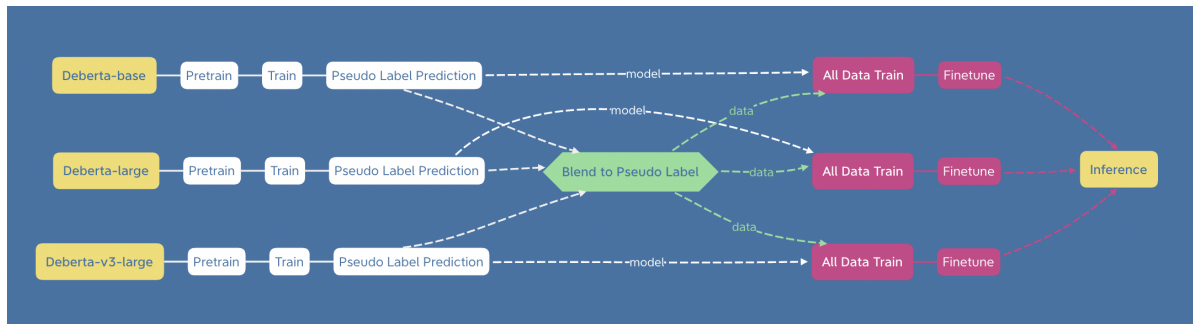
使用**全量数据 (14000多个含正式标签的样本 + 4万多个含伪标签的样本)**, 在第2步的训练结果上继续训练模型, 只需少量迭代次数即可。

5. Finetune 标签样本上微调

在第4步之后，在正式标签的样本上做少量迭代的微调，保存最终的3个模型的5个Fold（一共15个权重文件）

6. Inference 在kaggle上推理最终结果

将所有权重文件上传至Kaggle，然后根据base:0.15, large:0.25, v3-large:0.6的加权重blend融合，获得最终预测结果。



Dataset 代码

```
class NBMEDataset(Dataset):
    def __init__(self, tokenizer, df):
        self.tokenizer = tokenizer
        self.feature_texts = df['feature_text'].values # feature_text
        self.pn_historys = df['pn_history'].values # notes_text
        self.annotation_lengths = df['annotation_length'].values # annotation 数量
        self.locations = df['location'].values # annotation location

    def __len__(self):
        return len(self.feature_texts) # 长度

    def __getitem__(self, item):
        inputs = prepare_input(self.tokenizer,
                               self.pn_historys[item],
                               self.feature_texts[item])

        label = create_label(self.tokenizer,
                              self.pn_historys[item],
                              self.feature_texts[item],
                              self.annotation_lengths[item],
                              self.locations[item])

        return {'**inputs', 'label': label}
```

模型代码

```
# Model
class NBMEModel(nn.Module):
    def __init__(self, checkpoint):
        super().__init__()
        # output_hidden_states=True 返回所有层的隐藏状态。 返回值为 hidden_states。
        # hidden_states 是一个元组，它的第一个元素是Embedding，其余元素是各层的输出，
        shape == [bs, seq_len, hidden_size]
        self.config = AutoConfig.from_pretrained(checkpoint,
        output_hidden_states=True) # AutoConfig
        self.backbone = AutoModel.from_pretrained(checkpoint) # AutoModel
        self.dropout = nn.Dropout(0.1) # Dropout
        self.classifier = nn.Linear(self.config.hidden_size, 1) #MLP
        self._init_weights(self.classifier) # 初始化

    def _init_weights(self, module):
        if isinstance(module, nn.Linear):
            # Linear 层
            module.weight.data.normal_(mean=0.0,
            std=self.config.initializer_range)
            if module.bias is not None:
                module.bias.data.zero_()
        elif isinstance(module, nn.Embedding):
            # Embedding 层
            module.weight.data.normal_(mean=0.0,
            std=self.config.initializer_range) # initializer_range: 0.02
            if module.padding_idx is not None:
                # padding部分置零
                module.weight.data[module.padding_idx].zero_()
        elif isinstance(module, nn.LayerNorm):
            # Normalization层
            module.bias.data.zero_()
            module.weight.data.fill_(1.0)

    def forward(self, **inputs):
        #inputs(dict)
        # input_ids, token_type_ids, attention_mask, label
        outputs = self.backbone(**{k: v for k, v in inputs.items() if k !=
        'label'})
        # outputs: [last_hidden_state], last_hidden_state: [bs, seq_len,
        hidden_size]
        sequence_output = outputs[0]
        logits = self.classifier(self.dropout(sequence_output)) #获得 preds
        loss = None
        if 'label' in inputs:
            # 计算loss
            loss_fct = nn.BCEWithLogitsLoss(reduction="none")
            loss = loss_fct(logits.view(-1, 1), inputs['label'].view(-1,
            1).float())
            loss = torch.masked_select(loss, inputs['label'].view(-1, 1) !=
            -100).mean()
        # 返回值
        return TokenClassifierOutput(
            loss=loss, # loss
            logits=logits, # logits
        )
```

比赛上分历程

1. deberta-base Baseline 5Fold, Public LB : 0.868;
2. deberta-large Baseline 5Fold, Public LB : 0.873;
3. deberta-v3-large Baseline 5Fold, Public LB : 0.878;
4. 使用了Pretrain后, 并对模型进行调参 base:0.869, large:0.876, large:0.881;
5. 将3个模型 (15个权重文件) 进行融合, Public LB : 0.886;
6. 加入Pseudo Label全流程并调参, base:0.874, large:0.880, large:0.887;
7. 将3个模型 (15个权重文件) 进行融合, Public LB : 0.892;
8. 调整pseudo和inference阶段 blend加权重系数, Public LB : 0.893+;

代码、数据集

- 代码
 - 1.nbme-pretrain.ipynb
 - 2.nbme-train.ipynb
 - 3.nbme-pseudo-prediction.ipynb
 - 4.nbme-pseudo-blend.ipynb
 - 5.nbme-alldata-train.ipynb
 - 6.nbme-finetune.ipynb
 - 7.nbme-inference.ipynb
- 数据集
 - 官方数据集
 - train_processed.pkl # 有标签数据集
 - train_pl_all.pkl # 全量数据集

TL;DR

本次竞赛是需要我们将临床病人笔记文本中的表述和专业临床概念映射起来。在数据上我们修复了官方数据的不干净的原标签部分, 又因为本次比赛的最大特点是虽然比赛数据中只有少量包含label, 所以在我们使用了pseudo label来训练大量的无标注数据。模型上我们采用了 DeBERTa系列的3个模型融合: deberta-base, deberta-large, deberta-v3-large, 最后进行CV-5Fold和简单的加权融合。最终我们获得了Private LB: 0.883 (Top1%) 的成绩。